

ContactsFreeShare Mobile

A friendly, step-by-step guide to running the phone version on your own Google account — and adding it to your phone like a regular app.

This handbook walks you through everything from start to finish. You do not need to be a programmer. Read it like a short book: most chapters are easy, and the one technical chapter is broken into small, copy-and-paste steps you can follow slowly. Don't skip the boxes labeled *Heads-up* and *Tip* — they save you the most trouble.

CONTENTS

1. What the mobile app is, and what you'll need
2. Create a Google Cloud project and turn on billing
3. Create the permission slip (OAuth client)
4. Install the Google Cloud tool (gcloud)
5. Get the code and publish the app
6. Finish the Google permission settings
7. Add it to your phone
8. Daily use, costs, updates, and trouble
9. Quick reference

CHAPTER ONE

What the mobile app is, and what you'll need

The mobile version of ContactsFreeShare is a little different from the desktop app. There is **nothing to download from an app store**. Instead, the mobile app is a small private website that you publish into *your own* Google Cloud account. You then open it in your phone's web browser and "add it to your home screen," where it behaves just like a normal app with its own icon.

Once it's running, you simply sign in with your Google account, and the app lets you view and edit your contacts and pending changes right from your phone. If other family members sign in, each person sees only their own contacts — one published app can serve several people.

HEADS-UP: DIFFICULTY AND COST

Publishing the app (Chapters Two through Six) is the technical part. It involves Google Cloud, a credit card to switch on billing, and copying a few commands into a terminal window. Set aside one to two hours the first time. If you've never used a terminal, it's perfectly fine to ask a more technical friend to sit with you for those chapters.

The good news on cost: unlike the sharing server, the mobile app is allowed to "sleep" when no one is using it, so for personal and family use it usually stays within Google's free allowance — often costing little or nothing. You still must enable billing with a card for Google to allow it to run, and the very first open after it has been idle may take a few extra seconds to wake up.

What to have ready

You'll need a Google account, a computer (Mac or Windows) to do the one-time publishing, a smartphone to use the app on, and — to switch on Google Cloud — a credit or debit card. You'll also need the project's **source code folder** from the person who sent you this guide: the folder that contains a sub-folder named `mobilecontacts` .

TIP: ALREADY SET UP THE DESKTOP OR SHARING APP?

If you already created a Google Cloud project and an OAuth "permission slip" for the desktop or sharing app, you can reuse the very same project and client here. Just *add* the mobile app's web addresses to your existing client (Chapter Six) rather than creating new ones — and you can skip ahead to Chapter Four.

CHAPTER TWO

Create a Google Cloud project and turn on billing

Go to console.cloud.google.com and sign in with your Google account. Near the top of the page, open the project dropdown and choose **New Project**. Give it a simple name like `my-contacts` and click **Create**. Make a note of the **Project ID** shown under the name — it looks something like `my-contacts-123456` and you'll need it later.

Next, use the search bar at the top to find **Billing**, and link a billing account by adding a card. Google Cloud will not run the app without billing switched on, even though light personal use typically stays free.

Finally, search for **Google People API**, open it, and click **Enable**. This is what lets the app read and update your contacts.

TIP: CAP YOUR SPENDING

Under **Billing** → **Budgets & alerts** you can set a small monthly budget — say five dollars — so Google emails you if costs ever climb unexpectedly.

Create the permission slip (OAuth client)

Before the app can touch your contacts, Google needs you to create a small permission slip called an **OAuth client**. It's free and you only do it once.

1. In the Console, search for **OAuth consent screen**. Choose **External**, click **Create**, then fill in an app name and your own email where asked (support email and developer contact). Click through the **Scopes** page without changes, and on the **Test users** page add your own Google email address (and the emails of any family members who'll use it).
2. Search for **Credentials**, click **Create Credentials**, and choose **OAuth client ID**. Pick **Web application** as the type, and give it a name like `Mobile app`.
3. You'll fill in the web addresses in Chapter Six, after you know your app's real address. For now just click **Create**. A window shows your **Client ID** and **Client secret** — copy both somewhere safe.

HEADS-UP: THE SEVEN-DAY LIMIT

While the consent screen is in **Testing** mode, Google makes everyone sign in again every seven days. For smoother day-to-day use, open the **OAuth consent screen** later and click **Publish App** to move it to **In production**. You can do this without formal Google verification — you'll just click past an "unverified app" notice the first time — and it removes the weekly sign-in. It supports about a hundred users, which is plenty for a family.

CHAPTER FOUR

Install the Google Cloud tool (gcloud)

This is a small program that lets your computer talk to Google Cloud so it can publish the app for you.

On a Mac, open the **Terminal** app and run:

```
curl https://sdk.cloud.google.com | bash
```

Then close and reopen Terminal. (If you prefer a clickable installer, follow the steps at cloud.google.com/sdk/docs/install.)

On Windows, download and run the installer from that same page (look for the Windows installer). When it finishes, open the **Google Cloud SDK Shell** from the Start menu — use that window for every command in this guide.

Confirm it works by running:

```
gcloud --version
```

You should see version numbers rather than an error. Then sign in and choose your project, replacing `YOUR_PROJECT_ID` with the ID you noted in Chapter Two:

```
gcloud auth login  
gcloud config set project YOUR_PROJECT_ID
```

The first command opens a browser to confirm — sign in with the same Google account.

Get the code and publish the app

Step 1 — Put the source code on your computer

You need the project's source folder — the one that contains the `mobilecontacts` sub-folder. The person sharing this guide will give it to you as a `.zip` to unzip or a link to download. Put it somewhere easy to reach, such as your home folder, then move into the mobile sub-folder in the terminal. For example:

```
cd ~/PythonDB4-3-26/mobilecontacts
```

Use the actual name and location of the folder where you placed it.

Step 2 — Create the app's settings file

The app reads its settings from `python/.env`. Create it from the included example:

```
cd python  
cp .env.example .env
```

Now generate a random secret key. On Mac or Linux run the first command; on Windows (in the Google Cloud SDK Shell) run the second:

```
openssl rand -hex 32
```

```
python -c "import secrets; print(secrets.token_hex(32))"
```

Copy the long random string it prints. Then open `python/.env` in a plain text editor (TextEdit on Mac, Notepad on Windows) and fill in these three lines:

```
GOOGLE_CLIENT_ID=paste-your-client-id-here
GOOGLE_CLIENT_SECRET=paste-your-client-secret-here
APP_SECRET_KEY=paste-the-random-string-you-generated
```

HEADS-UP

Keep `APP_SECRET_KEY` safe and don't change it later — changing it signs everyone out. Leave the rest of the file as it is; the publishing script fills in the other settings automatically.

Step 3 — Publish (deploy) the app

Go back up to the `mobilecontacts` folder and run the deploy script. The extra `MOBILE_CUSTOM_DOMAIN=""` part is important — it tells the script to use *your* Google Cloud web address instead of a special domain you don't own:

```
cd ..
MOBILE_CUSTOM_DOMAIN="" bash scripts/deploy_cloud_run.sh
```

HEADS-UP: DON'T SKIP MOBILE_CUSTOM_DOMAIN=""

If you run the script without that part, it will try to use the address `mobile.contactsfreeshare.org`, which belongs to the original author — and sign-in will fail for you. Typing `MOBILE_CUSTOM_DOMAIN=""` in front of the command (exactly as shown) avoids this.

The first run takes several minutes. It switches on the Google Cloud services it needs, creates a storage area for everyone's data, and builds and publishes the app. If it asks you to confirm enabling anything, say **yes**. When it finishes, it prints something like:

```
Public URL: https://mobile-contacts-xxxxxxx-uc.a.run.app
Mobile UI:  https://mobile-contacts-xxxxxxx-uc.a.run.app/
```

Copy that `https://...run.app` **address.** It is the link you'll open on your phone, and you'll also paste it into Google's settings in the next chapter.

CHAPTER SIX

Finish the Google permission settings

Now that you know your app's real address, tell Google about it. In the Console, go to **Credentials** and click the Web-application OAuth client you made in Chapter Three. You need to add the address in *two* places on that page, replacing the example with *your* real address from Chapter Five:

Under **Authorized redirect URIs**, click **Add URI** and add:

```
https://mobile-contacts-xxxxxxx-uc.a.run.app/auth/callback
```

Under **Authorized JavaScript origins**, click **Add URI** and add the address with no path on the end:

```
https://mobile-contacts-xxxxxxx-uc.a.run.app
```

Click **Save**. If you're reusing a client that already has desktop, sharing, or other addresses listed, leave those in place — just add these two.

TIP

Web addresses must match exactly, character for character — including `https` and the `/auth/callback` path. The most common sign-in error comes from a small typo here.

CHAPTER SEVEN

Add it to your phone

Open your phone's web browser and go to your app's address (the `https://...run.app/` link from Chapter Five). Sign in with your Google account. Because this is a personal app you published yourself, Google shows an "unverified app" notice — that's expected; tap **Advanced**, then **Go to (your app name)**, and allow access.

Once it loads, add it to your home screen so it gets its own icon and opens full-screen like a normal app.

On an iPhone (Safari)

Make sure you're using **Safari**. Tap the **Share** button (the square with an upward arrow at the bottom of the screen), scroll down, and tap **Add to Home Screen**. Confirm the name and tap **Add**. A new icon appears on your home screen.

On an Android phone (Chrome)

Tap the **three-dot menu** in the top right, then tap **Add to Home screen** (or **Install app**). Confirm, and the icon appears among your apps.

TIP

From now on, open the app from that home-screen icon. The first time you open it after it's been idle for a while, it may take a few seconds to wake up — that's normal and only happens occasionally.

Daily use, costs, updates, and trouble

Everyday use and family members

Anyone you added as a Test user (or anyone at all, once you've published the app to production) can open the same link, sign in with their own Google account, and manage their own contacts. Each person's data is kept separate.

Watching the cost

Because the app sleeps when idle, personal use usually stays within Google's free allowance. Keep the budget alert from Chapter Two switched on so you're notified if that ever changes.

Updating later

When you receive a newer version of the source code, republish with the same command from the `mobilecontacts` folder:

```
MOBILE_CUSTOM_DOMAIN="" bash scripts/deploy_cloud_run.sh
```

Pausing or shutting down

To stop the app entirely, open **Cloud Run** in the Console, click the **mobile-contacts** service, and choose **Delete**. You can always republish later.

When something goes wrong

If the terminal says **"gcloud: command not found,"** the Google Cloud tool isn't installed or you didn't reopen the terminal afterward — redo Chapter Four with a fresh window. If the deploy script complains that **APP_SECRET_KEY** or **GOOGLE_CLIENT_ID** is required, the matching line in `mobilecontacts/python/.env` is still blank; fill it in and rerun.

If sign-in shows "**redirect_uri_mismatch**," the address isn't listed correctly under your OAuth client — re-check Chapter Six, and make sure you used `MOBILE_CUSTOM_DOMAIN=""` when deploying so the app's address matches the one you added. An "**unverified app**" warning is normal for a personal build; tap **Advanced** and continue. And if everyone is asked to sign in again about once a week, publish the consent screen to production as described in Chapter Three.

CHAPTER NINE

Quick reference

Keep this table handy — it collects every address and file path used in the guide.

What	Value
Deploy command (from mobilecontacts/)	MOBILE_CUSTOM_DOMAIN="" bash scripts/deploy_cloud_run.sh
Settings file	mobilecontacts/python/.env
Required settings	GOOGLE_CLIENT_ID , GOOGLE_CLIENT_SECRET , APP_SECRET_KEY
App address (after deploy)	https://mobile-contacts-xxxxxxx-uc.a.run.app/
OAuth redirect URI	<app-url>/auth/callback
OAuth JavaScript origin	<app-url> (no path)
Local testing address	http://127.0.0.1:8090
iPhone install	Safari → Share → Add to Home Screen
Android install	Chrome → menu → Add to Home screen

You did it. Keep this guide somewhere handy in case you ever need to republish, update, or help a friend through the same steps.